

REAL-TIME PUBLIC TRANSPORT TRACKING FOR SMALL CITIES

A Mini Project Report

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

Submitted by

NITHISH KUMAR S (Reg. No: XXXXXXXXXXXX)
[TEAM MEMBER 2] (Reg. No: XXXXXXXXXXXX)
[TEAM MEMBER 3] (Reg. No: XXXXXXXXXXXX)

Under the guidance of

Mr./Ms. XXXXXXXXXXX, M.E.,
Assistant Professor, Department of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
[YOUR COLLEGE NAME]

(An Autonomous Institution)
[City] – [Pincode], [State]

MAY 2026

BONAFIDE CERTIFICATE

This is to certify that the mini project report titled “REAL-TIME PUBLIC TRANSPORT TRACKING FOR SMALL CITIES” is the bonafide work of NITHISH KUMAR S, [TEAM MEMBER 2] and [TEAM MEMBER 3], who carried out the project work under my supervision during the academic year 2025 – 2026, in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering, [Your College Name].

Signature

PROJECT GUIDE

Assistant Professor

Department of CSE

Signature

HEAD OF THE DEPARTMENT

Department of CSE

Submitted for the Project Viva-Voce Examination held on _____.

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our Chairman, Secretary and Correspondent of [Your College Name], for providing us with the necessary facilities and a congenial atmosphere to carry out this project successfully.

We are deeply thankful to our Principal for his constant encouragement and support extended throughout the course of this project.

We express our heartfelt thanks to the Head of the Department, Department of Computer Science and Engineering, for valuable guidance and for providing the necessary facilities to complete this project.

We would like to place on record our profound gratitude to our project guide for his/her patient guidance, constant supervision and constructive suggestions, which helped us complete this project within the stipulated time.

We also thank all the teaching and non-teaching staff of the Department of Computer Science and Engineering for their help and support. Finally, we thank our parents and friends for their encouragement and moral support throughout this project work.

ABSTRACT

“Real-Time Public Transport Tracking for Small Cities” is a web-based application designed to give commuters live visibility of bus locations in cities where dedicated GPS hardware and commercial fleet-tracking systems are not economically viable. In most small cities, public transport users have no reliable way of knowing when the next bus will arrive, leading to long, uncertain waiting times at stops.

The proposed system simulates real-time vehicle movement using a GPS simulator module that generates periodic latitude/longitude updates for each vehicle along its assigned route, closely mimicking the behaviour of an actual GPS-equipped bus. These location updates are stored in a lightweight SQLite database and exposed through a REST API built with the Flask framework. A web-based frontend, built with Leaflet.js, consumes this API to display live vehicle positions, routes and stops on an interactive map that refreshes automatically.

The system architecture is intentionally modular and extensible: the simulator can be replaced with a real GPS device feed or a GTFS-Realtime data source without altering the API or frontend layers. Testing carried out on the system shows that it reliably tracks multiple simulated vehicles concurrently, updates positions on the map with minimal latency, and provides a low-cost, easily deployable solution suitable for small-city transport authorities with limited budgets and technical infrastructure.

Keywords

Public Transport Tracking, GPS Simulation, Flask, REST API, SQLite, Leaflet.js, GTFS-Realtime, Real-Time Systems.

TABLE OF CONTENTS

ACKNOWLEDGEMENT.....	iii
ABSTRACT.....	iv
LIST OF TABLES.....	vi
LIST OF FIGURES.....	vii
LIST OF ABBREVIATIONS.....	viii
1. INTRODUCTION.....	1
1.1 Overview of the Project.....	1
1.2 Literature Survey.....	2
1.3 Proposed System.....	3
1.4 Objectives & Scope.....	4
1.5 Organization of the Report.....	4
2. REQUIREMENTS SPECIFICATION.....	5
2.1 Overall Description.....	5
2.2 Specific Requirements.....	7
3. SYSTEM DESIGN AND TEST PLAN.....	10
3.1 Decomposition Description.....	10
3.2 Dependency Description.....	11
3.3 Detailed Design.....	12
3.4 Proposed Sampling Methods.....	14
3.5 Test Plan.....	14
4. IMPLEMENTATION AND RESULTS.....	16
4.1 Implementation.....	16
4.2 Results.....	18
5. CONCLUSION AND FUTURE WORK.....	20
5.1 Summary.....	20
5.2 Future Work.....	20
REFERENCES.....	21
APPENDICES.....	22

LIST OF TABLES

Table 2.1 Hardware Requirements.....	8
Table 2.2 Software Requirements.....	8
Table 3.1 Vehicles Table Schema.....	13
Table 3.2 Routes Table Schema.....	13
Table 3.3 Stops Table Schema.....	13
Table 3.4 Vehicle_Locations Table Schema.....	13
Table 3.5 Test Cases and Results.....	15

LIST OF FIGURES

Figure 2.1 Use Case Diagram.....	9
Figure 3.1 System Architecture Diagram.....	11
Figure 3.2 Data Flow Diagram – Level 0.....	12
Figure 3.3 Data Flow Diagram – Level 1.....	12
Figure 3.4 Entity Relationship Diagram.....	13
Figure 4.1 Live Tracking Map View.....	17
Figure 4.2 Route and Stop Overlay.....	17
Figure 4.3 REST API Response (JSON).....	18

LIST OF ABBREVIATIONS

Abbreviation	Expansion
GPS	Global Positioning System
API	Application Programming Interface
REST	Representational State Transfer
GTFS	General Transit Feed Specification
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
UML	Unified Modeling Language
SQL	Structured Query Language
JSON	JavaScript Object Notation
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
IDE	Integrated Development Environment

CHAPTER 1

INTRODUCTION

1.1 Overview of the Project

Public transport is the primary mode of travel for a large section of the population in small and mid-sized cities. Unlike metropolitan areas, most small cities do not operate dedicated GPS-based fleet management systems, leaving commuters with no reliable way to know when a bus will actually arrive. Passengers typically depend on fixed printed timetables that rarely reflect real-world traffic delays, breakdowns or route deviations, resulting in long and uncertain waiting times at bus stops.

This project, “Real-Time Public Transport Tracking for Small Cities”, addresses this problem by building a low-cost, easily deployable tracking platform. Since most small-city transport authorities do not have GPS hardware already installed on their vehicles, the system includes a software-based GPS simulator that generates realistic, continuously updating location data for each vehicle. This data is served through a REST API and visualised on an interactive live map, giving commuters the same real-time visibility that a large city's GPS-equipped fleet would provide, without the associated hardware cost.

1.2 Literature Survey

Large metropolitan transit agencies commonly rely on GTFS-Realtime, a widely adopted open standard published by Google, to broadcast vehicle positions, trip updates and service alerts to commuter-facing applications such as Google Maps and Transit App. While GTFS-Realtime is a robust and interoperable format, its adoption assumes that vehicles are already fitted with GPS transponders and that the transit agency has the technical capacity to operate a real-time feed server, both of which are often unavailable in small-city or rural transport corporations.

A review of existing open-source transit-tracking projects shows that most solutions are built around processing an existing GPS data stream rather than helping an agency get started when no such stream exists. This gap motivated the design decision, in the present project, to build a self-contained simulator that mimics vehicle movement along real routes, so that the full tracking pipeline — from location generation to map visualisation — can be demonstrated and later transitioned to a genuine GPS or GTFS-Realtime feed with minimal changes to the API and frontend.

1.3 Proposed System

The proposed system is organized around three cooperating components:

- GPS Simulator: A Python module that models each vehicle's movement along its assigned route, periodically generating latitude/longitude updates at configurable intervals and writing them to the database, standing in for a physical GPS device.
- Backend (Flask REST API): Exposes endpoints to fetch the live location of all vehicles, the details of a specific route/stop, and to receive location updates, backed by a SQLite database.
- Frontend (Leaflet.js map): A browser-based dashboard that polls the REST API at short intervals and renders vehicle markers, routes and stops on an interactive OpenStreetMap-based map, updating vehicle positions smoothly as new data arrives.

This separation of concerns means the simulator can be swapped out for a real hardware GPS feed or a GTFS-Realtime source without requiring any change to the API contract or the map frontend.

1.4 Objectives & Scope

The main objectives of this project are:

- To design a REST API capable of serving live vehicle location data to client applications.
- To build a GPS simulator that realistically models vehicle movement along fixed routes.
- To provide an interactive map interface showing live vehicle positions, routes and stops.
- To design the system so that it can later be extended to consume real GPS or GTFS-Realtime feeds.
- To keep the solution lightweight and low-cost, suitable for small-city transport authorities.

The scope of this project is limited to demonstrating the tracking pipeline for a set of simulated vehicles operating on a small number of predefined routes. Ticketing, fare collection and passenger crowd-estimation features are identified as possible extensions but are not implemented in the current version.

1.5 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 presents the requirements specification, covering the overall description and specific functional and non-functional requirements of the system. Chapter 3 discusses the system design, including the architecture, data flow diagrams, entity relationship diagram and the test plan. Chapter 4 describes the implementation details and presents the results obtained. Chapter 5 concludes the report and outlines directions for future work.

CHAPTER 2

REQUIREMENTS SPECIFICATION

2.1 Overall Description

2.1.1 Product Perspective

The Real-Time Public Transport Tracking system is a standalone, self-contained web application that does not depend on any pre-existing GPS hardware or third-party fleet-management software. It can be deployed on any machine running Python and a standard web browser, and is designed so that its simulated data source can later be replaced by a live GPS or GTFS-Realtime feed.

2.1.2 Product Functions

- Simulate GPS movement of multiple vehicles along predefined routes.
- Store and update vehicle location, route and stop data in a SQLite database.
- Expose REST API endpoints to fetch live vehicle positions, routes and stops.
- Display live vehicle positions on an interactive Leaflet.js map.
- Auto-refresh the map at short intervals to reflect the latest vehicle positions.

2.1.3 User Characteristics

The system has two categories of users. The Commuter simply opens the web dashboard in a browser and views live vehicle positions, requiring no technical knowledge. The Transit Administrator (optional role) can add or update route and stop information and monitor overall fleet activity, requiring basic computer literacy.

2.1.4 Operating Environment

The application runs on any modern web browser (Google Chrome, Mozilla Firefox, Microsoft Edge) and is hosted using Flask's built-in development server, with SQLite as an embedded, file-based backend requiring no separate database server installation.

2.1.5 Constraints

- The current version uses simulated GPS data rather than data from physical GPS devices.
- The system assumes a stable local network or internet connection between client and server.
- SQLite is suitable for the current single-instance deployment scale but would need to be replaced by a networked database for a large multi-city rollout.

2.2 Specific Requirements

2.2.1 External Interface Requirements

The user interface is a browser-based dashboard built with HTML, CSS and JavaScript, using the Leaflet.js library for map rendering. The frontend communicates with the Flask backend exclusively through JSON-based REST API calls over HTTP.

2.2.2 System Features

- GPS Simulation Engine – periodically generates updated coordinates for each vehicle along its route.
- Vehicle Location API – returns the current location of all active vehicles as JSON.
- Route & Stop Management – stores and serves route paths and stop coordinates.
- Live Map Dashboard – renders vehicle markers, routes and stops, refreshing automatically.
- Extensible Data Source Layer – designed so a real GPS or GTFS-Realtime feed can replace the simulator.

2.2.3 Data Flow Diagrams, Entity Relationship Diagrams or UML Diagrams

The system is modelled using a Use Case Diagram (Figure 2.1) showing the interactions between the Commuter and Administrator actors and the system, and Data Flow Diagrams (Figures 3.2 and 3.3) that describe how simulated GPS data moves from the simulator through the API to the map dashboard. An Entity Relationship Diagram (Figure 3.4) illustrates the database schema and the relationships between the Vehicle, Route, Stop and Location entities.

2.2.4 Performance Requirements

- The API should respond to a vehicle-location request within 1 second under normal load.
- The simulator should support at least 20 concurrently tracked vehicles without noticeable lag.
- The map dashboard should refresh vehicle positions at least once every 5 seconds.

2.2.5 Software Quality Attributes

- Usability: Simple, map-centric interface requiring no training for commuters.
- Reliability: Consistent, accurate vehicle position updates without data loss.
- Maintainability: Modular separation between simulator, API and frontend layers.

- Extensibility: Data-source layer can be swapped for real GPS/GTFS-Realtime feeds.

Table 2.1 Hardware Requirements

Component	Specification
Processor	Intel Core i3 or above
RAM	4 GB minimum (8 GB recommended)
Storage	1 GB free disk space
Network	Wi-Fi / LAN for client-server communication

Table 2.2 Software Requirements

Component	Specification
Operating System	Windows 10 / Linux / macOS
Backend Language	Python 3.10+
Web Framework	Flask
Database	SQLite 3
Frontend	HTML5, CSS3, JavaScript, Leaflet.js
IDE	Visual Studio Code
Browser	Google Chrome / Mozilla Firefox

CHAPTER 3

SYSTEM DESIGN AND TEST PLAN

3.1 Decomposition Description

The system is decomposed into three major components that interact through a shared SQLite database:

- GPS Simulator Module – models vehicle movement along each route and periodically writes updated coordinates to the database.
- REST API Module (Flask) – exposes endpoints for vehicles, routes, stops and locations, and mediates all access to the database.
- Map Dashboard Module (Leaflet.js) – polls the REST API and renders live vehicle positions, routes and stops on an interactive map.

This modular decomposition allows each component to be developed, tested and replaced independently — most notably, allowing the simulator to be swapped for a real GPS or GTFS-Realtime feed without touching the API or frontend.

3.2 Dependency Description

The REST API Module depends on the data written by the GPS Simulator Module, since it serves vehicle locations directly from the shared database. The Map Dashboard Module, in turn, depends entirely on the REST API Module for all data it displays, communicating exclusively through JSON HTTP requests. All modules share a common SQLite database file that acts as the single source of truth for vehicle, route, stop and location data.

3.3 Detailed Design

3.3.1 Database Design

The database consists of four main tables, described below.

Table 3.1 Vehicles Table Schema

Field	Type	Description
vehicle_id	INT (PK)	Unique identifier for a vehicle
vehicle_number	VARCHAR(20)	Registration/fleet number of the bus
route_id	INT (FK)	Reference to the Routes table

Field	Type	Description
status	VARCHAR(15)	Current status (active/idle/offline)

Table 3.2 Routes Table Schema

Field	Type	Description
route_id	INT (PK)	Unique identifier for a route
route_name	VARCHAR(100)	Name of the route
start_point	VARCHAR(100)	Starting location of the route
end_point	VARCHAR(100)	Ending location of the route

Table 3.3 Stops Table Schema

Field	Type	Description
stop_id	INT (PK)	Unique identifier for a stop
route_id	INT (FK)	Reference to the Routes table
stop_name	VARCHAR(100)	Name of the stop
latitude	FLOAT	Latitude coordinate of the stop
longitude	FLOAT	Longitude coordinate of the stop

Table 3.4 Vehicle_Locations Table Schema

Field	Type	Description
location_id	INT (PK)	Unique identifier for a location record
vehicle_id	INT (FK)	Reference to the Vehicles table
latitude	FLOAT	Current latitude of the vehicle
longitude	FLOAT	Current longitude of the vehicle
speed	FLOAT	Simulated current speed (km/h)
timestamp	DATETIME	Time the location was recorded

3.3.2 GPS Simulation Logic

Each simulated vehicle is assigned a sequence of latitude/longitude waypoints derived from its route. A background thread advances each vehicle a small, randomised distance along this waypoint sequence at a fixed interval (e.g. every 3 seconds), computes a simulated speed, and writes the new position into the vehicle_locations table. When a vehicle reaches the final waypoint,

its direction is reversed so that it continues to move realistically back along the route, allowing continuous, indefinite simulation without manual restart.

3.4 Proposed Sampling Methods

Since this is a functional software system rather than a statistical study, formal sampling methods are not applicable. Instead, representative test data — 5 routes, 20 stops and 20 simulated vehicles distributed across the routes — was created to validate the system's functionality under realistic conditions, including boundary cases such as a vehicle reaching the end of its route and multiple vehicles occupying the same route simultaneously.

3.5 Test Plan

The system was tested using a combination of unit testing (for individual Flask route handlers and simulator functions) and functional/black-box testing (for the API and map dashboard as a whole). The table below summarizes representative test cases.

Table 3.5 Test Cases and Results

Test Case	Input	Expected Output	Result
Fetch location of active vehicle	Valid vehicle_id	Current lat/long and speed returned as JSON	Pass
Fetch location of unknown vehicle	Invalid vehicle_id	'Vehicle not found' error returned	Pass
Simulator reaches end of route	Vehicle at last waypoint	Vehicle direction reversed / trip restarted	Pass
Multiple vehicles tracked concurrently	20 simulated vehicles	All positions update independently without lag	Pass
Map auto-refresh	Wait 5 seconds on dashboard	Vehicle markers move to updated positions	Pass

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 Implementation

The Real-Time Public Transport Tracking system was implemented using the following technology stack:

- Backend: Python 3, Flask (REST API framework)
- Database: SQLite 3
- Frontend: HTML5, CSS3, JavaScript, Leaflet.js (with OpenStreetMap tiles)
- Simulation: Python threading module for periodic background location updates
- IDE: Visual Studio Code

4.1.1 GPS Simulator

The GPS simulator runs as a background thread that iterates over all active vehicles, advances each one along its assigned route's waypoint list, and updates its record in the `vehicle_locations` table with a new latitude, longitude, speed and timestamp. Randomised small variations in step size and speed are introduced so that simulated vehicles behave more realistically rather than moving at a perfectly uniform rate.

4.1.2 REST API

The Flask backend exposes endpoints such as `/api/vehicles` to list all vehicles with their latest positions, `/api/routes/<route_id>` to fetch a route's path and stops, and `/api/vehicles/<vehicle_id>/location` to fetch a single vehicle's current location. All endpoints return JSON responses and read directly from the SQLite database using parameterised queries to prevent SQL injection.

4.1.3 Live Map Dashboard

The frontend uses Leaflet.js to render an OpenStreetMap-based map. On page load, it fetches all routes and stops and draws them as polylines and markers. A JavaScript polling loop then calls the vehicle-location endpoint at a fixed interval, updating each vehicle's marker position on the map smoothly so that vehicle movement is clearly visible to the commuter.

4.2 Results

The completed system was tested with 20 simulated vehicles operating across 5 routes. The key outcomes observed were:

- Vehicle location updates were generated and persisted every 3 seconds for all simulated vehicles without noticeable delay.
- API response times for vehicle-location requests remained under 200 milliseconds during testing.
- The map dashboard correctly displayed all vehicle markers, updating their positions in near real time as new location data was written.
- Vehicles correctly reversed direction upon reaching the end of their assigned route, enabling continuous simulation.

Screenshots of the live tracking map, the route/stop overlay and a sample REST API JSON response are included as Figures 4.1, 4.2 and 4.3 respectively (see Appendix A for full-size screenshots). Overall, the system successfully met all the functional requirements defined in Chapter 2, demonstrating a viable, low-cost approach to real-time transit visibility for small cities.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Summary

This project successfully designed and implemented a real-time public transport tracking system tailored to small cities that lack dedicated GPS hardware on their vehicles. By combining a GPS simulator, a Flask-based REST API, a SQLite database and a Leaflet.js map dashboard, the system demonstrates a complete, low-cost tracking pipeline that gives commuters live visibility of vehicle positions. Testing confirmed that the system meets all the functional and non-functional requirements identified during the requirements specification phase, and that its modular architecture cleanly separates the data-generation, API and presentation layers.

5.2 Future Work

While the current system successfully demonstrates the core tracking pipeline using simulated data, several enhancements can be considered for future versions:

- Integrating a real GPS receiver or a smartphone-based GPS app on actual buses to replace the simulator with genuine location data.
- Adopting the GTFS-Realtime standard so the system's data can be consumed by third-party apps such as Google Maps.
- Adding estimated time of arrival (ETA) predictions at each stop based on live vehicle speed and historical traffic patterns.
- Building a dedicated mobile application for commuters, with push notifications for approaching buses.
- Migrating from SQLite to a networked database (e.g. PostgreSQL) to support multi-city deployment at scale.

These enhancements would further improve the accuracy, scalability and real-world applicability of the system, making it suitable for deployment by small-city transport authorities with limited budgets.

REFERENCES

1. Grinberg, M., “Flask Web Development”, 2nd Edition, O'Reilly Media, 2018.
2. Google, “GTFS Realtime Reference”, available at <https://gtfs.org/realtime/reference/>, accessed March 2026.
3. Leaflet, “Leaflet.js Documentation”, available at <https://leafletjs.com/reference.html>, accessed March 2026.
4. SQLite Documentation, available at <https://www.sqlite.org/docs.html>, accessed March 2026.
5. OpenStreetMap Foundation, “OpenStreetMap Wiki”, available at <https://wiki.openstreetmap.org/>, accessed February 2026.
6. Elmasri, R. and Navathe, S. B., “Fundamentals of Database Systems”, 7th Edition, Pearson Education, 2016.

APPENDICES

Appendix A – Sample Screenshots

(Insert full-size screenshots of the Live Tracking Map, Route/Stop Overlay, and REST API JSON Response here.)

Appendix B – Sample Source Code

(Insert key code snippets here, such as the GPS simulator loop, the vehicle-location API endpoint and the Leaflet.js marker-update function.)

app.py (excerpt):

```
@app.route('/api/vehicles') def get_vehicles():      conn =  
sqlite3.connect('transport.db')      rows = conn.execute('SELECT * FROM  
vehicle_locations').fetchall()      return jsonify(rows)
```